How-To Library

Informatica

Big Data Management 10.1.1 Performance Tuning for the Spark Engine

© Copyright Informatica LLC 2017, 2022. Informatica, the Informatica logo, and Big Data Management are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html

Abstract

This article describes general reference guidelines and best practices to help you tune the performance of the Spark run-time engine. It contains information about best practices that you can implement when you enable dynamic resource allocation on the Spark run-time engine. It also discusses workarounds and troubleshooting tips for common issues.

Supported Versions

- Big Data Management® 10.1.1

Table of Contents

Overview. . . . 2
Hardware Recommendations. . . . 2
Setting Up the Spark History Server. . . . 3
Cluster-Side Configuration. . . . 3
Client-Side Configuration. . . . 4
Transformation Optimization. . . . 4
Configuring Performance Properties. . . . 5
Troubleshooting Job Failures for Large Data Volumes. . . . 7
Acknowledgements. . . . 7

Overview

You can improve Spark engine performance by using a technique called dynamic resource allocation. With dynamic resource allocation, you can adaptively distribute resources to maximize network performance metrics.

With traditional static resource allocation, resources are distributed in a predetermined manner. The Spark application fails to fully optimize CPU and memory, so these resources often go unused. Dynamic resource allocation is a nonlinear optimization technique that allocates server resources based on estimated application requirements.

This article outlines hardware prerequisites in addition to client-level and cluster-level configuration changes that are required for dynamic resource allocation on the Spark engine. It also includes workarounds and troubleshooting tips for common issues.

Hardware Recommendations

Informatica provides hardware recommendations to help you optimize Spark engine performance.

The following table lists minimum and optimal hardware recommendations for Spark:

Hardware	Minimum Recommendation	Optimal Recommendation
CPU speed	2-2.5 GHz	2.5-3.5 GHz
Total system CPU cores	16	24

Hardware	Minimum Recommendation	Optimal Recommendation
Total system CPU Vcores	32	48
Total system memory	32 GB	128 GB
Local disk	500 GB	2.4 TB
Number of nodes	4	12
HDFS replication factor	3	3
DFS block size	128 MB	256 MB
Disk capacity	250 GB	1200 GB
Number of disks for HDFS	8	12
Total available HDFS capacity per node	2 TB	14.4 TB
Total HDFS capacity on the cluster	8 TB	144 TB
Actual HDFS capacity with replication	2.66 TB	57.6 TB
`/tmp` mount point	20 GB	30 GB
Installation disk space requirement	12 GB	12 GB
`ulimit` settings	Open files: 32K Maximum number of processes: 8192 Core file size: unlimited	Open files: 64K Maximum number of processes: 32768 Core file size: unlimited
Network bandwidth (ethernet card)	Bonded channel (at least 2 Gbps)	10 Gbps ethernet card

Setting Up the Spark History Server

The Spark History Server is a monitoring tool that lists information about completed Spark applications. Before you enable dynamic resource allocation, you need to set up the server.

Cluster-Side Configuration

Start the History Server on the cluster. For detailed instructions, visit the following links based on your environment:

- Cloudera
- Hortonworks
- MapR
- Azure HD Insight

- Amazon EMR
- IBM BigInsights

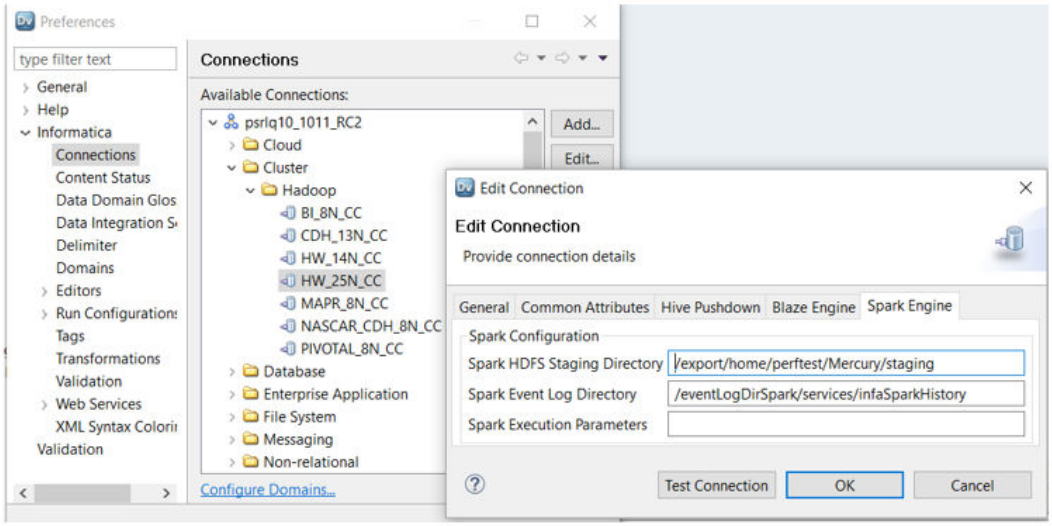
Client-Side Configuration

1. In the Developer tool, click **Window** > **Preferences** > **Informatica** > **Connections**.
2. Expand the domain in the **Available Connections** list.
3. Select the connection in **Available Connections**, and click **Edit**.

   The **Edit Connection** dialog box appears.
4. Click the **Spark Engine** tab.
5. Locate the following property in the Spark Application History Server: `spark.eventLog.dir`.
6. In the **Edit Connection** dialog box, configure the **Spark Event Log Directory** with the file path specified for `spark.eventLog.dir`.

   The Spark Event Log Directory is the base directory that logs Spark events. Within this base directory, Spark creates a subdirectory for each application, and logs the events specific to the application in this directory.

   The following image shows the completed **Edit Connection** dialog box:



7. To test the connection, click **Test Connection**.
8. Click **OK**.

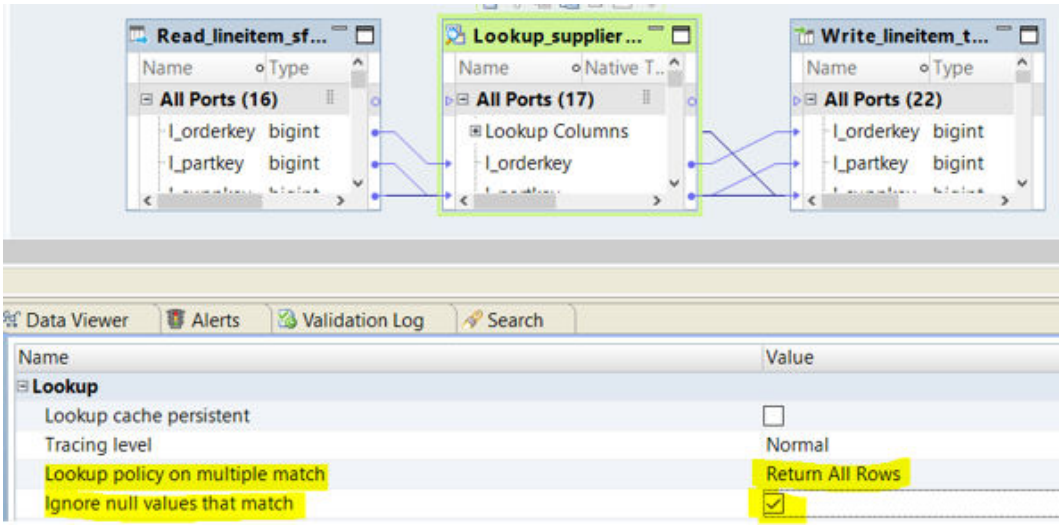
Transformation Optimization

This section provides information on optimizing transformation performance on the Spark engine.

Lookup Transformation

A Lookup transformation with NULL comparison check translates to a non equi join because it uses a comparison operator other than the equal sign for the condition. Non equi join performance is slow because it results in a Cartesian join.

By default, NULL comparison check is enabled. To optimize performance, disable the NULL comparison check by setting the following properties:



Router Transformation

The Spark engine stages data at the Router transformation, which slows performance.

To optimize performance, use the Blaze execution engine when a mapping contains a Router transformation.

Joiner Transformation

If there is not enough memory for a full outer join in a Joiner transformation, follow a two-step tuning process:

1. Ensure that every executor core has at least 3 GB of memory.
   For example, set `spark.executor.memory=6 GB` and `spark.executor.cores=2`.
2. Set shuffle partitions = <master partitions> + <detailed partitions>.
   For example, with a DFS block size of 256 MB, 100 GB of master data will have 400 splits and 200 GB of details will have 800 partitions.

   The recommended setting is `infaspark.shuffle.max.partitions=1200`.

Non Equi Join

Non equi join performance is slow because it uses an operator other than the equal sign (for example, >, <, >=, <=) as a comparison operator for the condition.

To optimize performance, use the Blaze execution engine when a mapping contains a non equi join.

Configuring Performance Properties

To enable dynamic resource allocation on the Spark engine, you must configure the following properties within the Hadoop properties file `hadoopEnv.properties`. The file is located in the following directory: `<Informatica installation directory>/services/shared/hadoop/<Hadoop distribution name>/infaConf`.

spark.dynamicAllocation.enabled

Enables dynamic resource allocation. This property is required when you enable the external shuffle service.

Recommended value: **TRUE**

spark.shuffle.service.enabled

Enables the external shuffle service. This property is required when you enable dynamic resource allocation.

Recommended value: **TRUE**

infaspark.shuffle.max.partitions

Sets the number of shuffle partitions to the maximum number of partitions seen across all input sources.

Recommended value: Allocate approximately **eight** dynamic shuffle partitions for each gigabyte of shuffle data.

For example, for 400 GB of shuffle data, set this value to 3200.

spark.sql.shuffle.partitions

Sets the number of shuffle partitions to a default value of 100.

Recommended value: Allocate approximately **eight** shuffle partitions for each gigabyte of shuffle data.

For example, for 100 GB of shuffle data, set this value to 800.

spark.sql.autoBroadcastJoinThreshold

Sets the map join threshold to a default value of 10 MB.

Recommended value: **256 MB**

spark.sql.statistics.fallBackToHdfs

Any Hive source with a data size less than the configured threshold will participate in map-side join if table statistics are available. When statistics are not available from the Hive metastore, you can force Spark to rely on HDFS statistics with this custom property.

Recommended value: **TRUE**

spark.driver.memory

Sets the driver process memory to a default value of 1 GB. The driver requires more memory based on the number of data sources and data nodes.

Recommended value: Allocate at least **256 MB** for every data source participating in map join.

For example, if a mapping has eight data sources, set the driver memory to at least 2 GB (i.e. 8 x 256).

spark.scheduler.maxRegisteredResourcesWaitingTime

Controls the Spark executor initialization time. You can reduce this time for small jobs where the initialization time is predominant.

Recommended value: **15000 milliseconds**

spark.executor.instances

Performance configuration to run mappings on the Spark engine. If you enable dynamic resource allocation for the Spark engine, Informatica recommends that you convert this property to a comment.

For example, `#spark.executor.instances=100`

Troubleshooting Job Failures for Large Data Volumes

This section provides information on troubleshooting common error messages and limitations that you might encounter when you enable dynamic resource allocation on the Spark engine. These errors might occur when you are processing a large volume of data, such as 10 TB or more, or when a job has a large shuffle volume.

- **Could not find CoarseGrainedScheduler.**
  When you stop a process, you might lose one or more executors with the following error:
  `cluster.YarnScheduler: Lost executor 8 on myhost1.com: remote Rpc client disassociated`

  One of the most common reasons for executor failure is insufficient memory. When an executor consumes more memory than the maximum limit, YARN causes the executor to fail. By default, Spark does not set an upper limit for the number of executors if dynamic allocation is enabled (SPARK-14228).

  To limit the number of executors, open the `hadoopEnv.properties` file and configure the property `Spark.dynamicAllocation.maxExecutors`. Determine the value based on available cores and memory per node.

  You can also configure the property `spark.executor.memory` to increase the amount of memory per executor process. The default value is 1 GB.

- **Total size of serialized results is bigger than spark.driver.maxResultSize.**
  The default maximum size of serialized results of all partitions for each Spark action is 1 GB. Jobs will be aborted if the total size is above this limit. Having a high limit may cause out-of-memory errors in the driver. Setting a proper limit can protect the driver from out-of-memory errors.

  To increase the size limit, open the `hadoopEnv.properties` file and configure the property `spark.driver.maxResultSize`. Set the result size for the driver to at least 2 GB, or 0 for unlimited.

- **java.util.concurrent.TimeoutException; Futures timed out after [300 seconds].**
  The default broadcast timeout limit is set to 300 seconds.

  To increase the SQL broadcast timeout limit, open the `hadoopEnv.properties` file and configure the property `spark.sql.broadcastTimeout`. Set the timeout limit to at least 600 seconds.

- **A job fails due to Spark speculative execution of tasks.**
  If one or more tasks are running slowly in a stage, they will be re-launched.

  To resolve this issue, open the `hadoopEnv.properties` file and configure the property `spark.speculation=FALSE`.

- **ShuffleMapStage 12 (rdd at InfaSprk1.scala:48) has failed the maximum allowable number of times: 4.**
  The Spark shuffle service fails because the GC overhead limit was exceeded. This forces the Node Manager to shut down, which eventually causes the Spark job to fail.

  To resolve this issue, open the YARN node manager and increase the maximum heap size in MB in the **NodeManager Java heap size** property.

  For further debugging, check the Node Manager logs:

  ```
  java.lang.OutOfMemoryError: GC overhead limit exceeded
  2016-12-07 19:38:29,934 FATAL
  yarn.YarnUncaughtExceptionHandler(YarnUncaughtExceptionHandler.java:uncaughtException
  51)) - Thread Thread[IPC Server handler 0 on 8040,5,main] threw an Error. Shutting down
  now...
  ```

Acknowledgements

The author would like to acknowledge Vishal Kamath, QA Engineering Manager, and Mohammed Morshed, Principal QA Engineer, for their technical assistance.

Author

Elizabeth Snyder